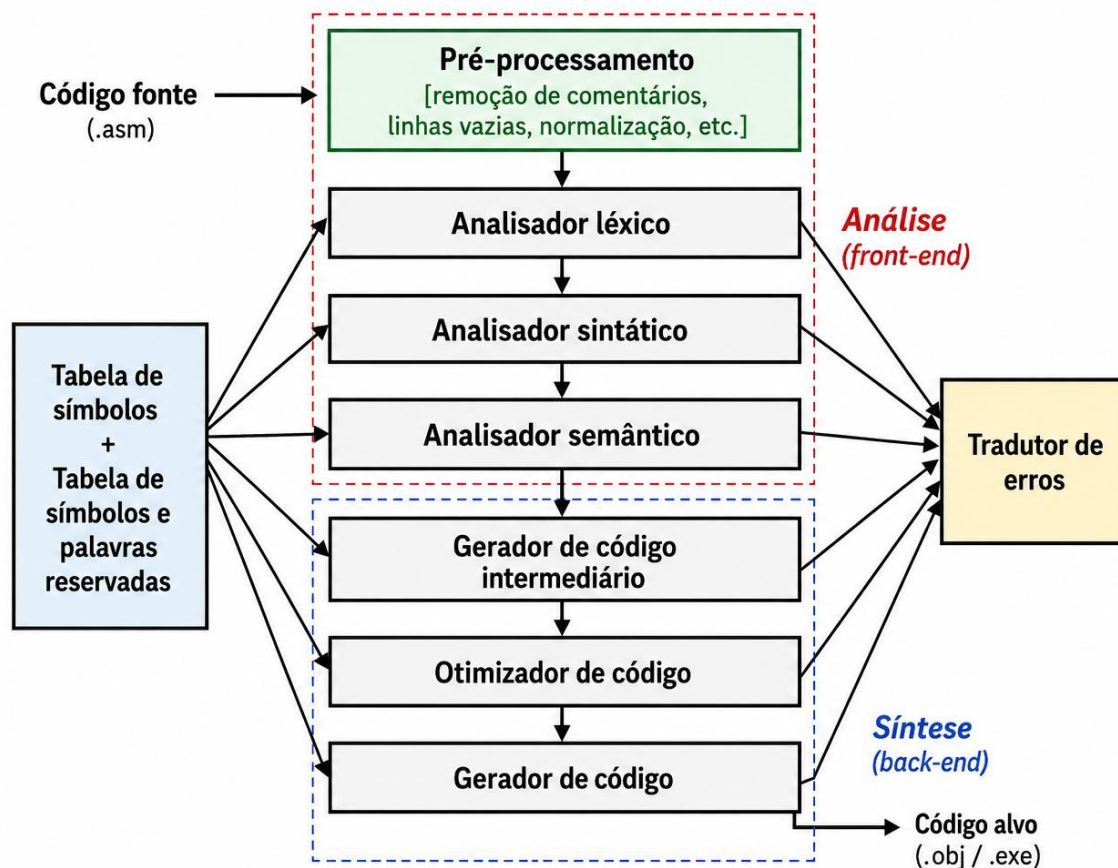


MANUAL DA LINGUAGEM μ -ASSEMBLY — MIPS/MARS

1. Apresentação

A μ -Assembly é a linguagem utilizada no projeto da disciplina de *Linguagens Formais, Autômatos e Compiladores* e corresponde a um subconjunto da linguagem **Assembly MIPS**, tomando como referência a sintaxe utilizada pelo simulador MARS (MIPS Assembler and Runtime Simulator).

O objetivo deste manual é estabelecer de maneira precisa quais construções serão aceitas pela linguagem ao longo das diferentes etapas do projeto. Dessa forma, o documento funcionará como uma especificação da linguagem, servindo de referência para o desenvolvimento do pré-processador, do analisador léxico, do analisador sintático, do analisador semântico e das etapas posteriores do tradutor.



1.1. Estrutura

A estrutura geral de um programa μ -Assembly pode ser compreendida como a organização do código em duas grandes seções: uma destinada aos dados e outra destinada às instruções executáveis. Essa organização segue o padrão utilizado nos exemplos do projeto, nos quais `.data` identifica a área de dados e `.text` marca o início da área de código.

```
.data
    # declaração de dados

.text
main:
    # instruções
```

A diretiva `.data` indica o início da seção de dados do programa. Nessa região são declaradas informações que deverão permanecer armazenadas na memória durante a execução, como números inteiros e strings. Essas declarações normalmente são associadas a rótulos, que funcionam como nomes simbólicos para facilitar o acesso posterior aos dados.

Por exemplo:

```
.data
mensagem: .ascii "Resultado: "
valor: .word 10
```

Nesse trecho, `mensagem` e `valor` são rótulos. O primeiro identifica uma string declarada por meio de `.ascii`, enquanto o segundo identifica uma palavra de memória inicializada com o valor 10 por meio de `.word`. Essa forma de organização aparece expressamente nos exemplos do enunciado.

A diretiva `.text` marca o início da seção que contém as instruções do programa. É nessa parte que ficam as operações que serão executadas pelo processador, como carregamento de valores em registradores, operações aritméticas e desvios. No projeto, o rótulo `main:` é utilizado para identificar o início do trecho principal de execução.

Um exemplo mais completo é:

```
.data
mensagem: .ascii "Resultado: "
valor: .word 10

.text
main:
    li $t0, 5
    li $t1, 10
    add $t2, $t0, $t1
```

Nesse programa, a seção `.data` contém os dados armazenados em memória, enquanto a seção `.text` contém as instruções. A pseudoinstrução `li $t0, 5` carrega o valor 5 em `$t0`, `li $t1, 10` carrega o valor 10 em `$t1`, e `add $t2, $t0, $t1` soma os valores e armazena o resultado em `$t2`.

Uma forma mais detalhada de representar a estrutura é:

```
1  .data
2  # declaração de dados
3
4  mensagem:  .asciiz  "Resultado: \n"
5  valor:     .word    10
6  vetor:     .word    1, 2, 3, 4, 5
7  tamanho:   .word    5
8
9  # outros dados podem ser declarados aqui
10
11 .text
12 # início da seção de código (instruções)
13
14 main:
15 # início do programa principal
16 la  $a0, mensagem  # carrega endereço da string
17 li  $v0, 4          # código para imprimir string
18 syscall            # chamada ao sistema
19
20 li  $t0, 5
21 li  $t1, 10
22 add $t2, $t0, $t1  # $t2 = $t0 + $t1
```

SEÇÃO DE DADOS

Inicia com a diretiva `.data`.
Contém as declarações de dados (números inteiros, strings, vetores, etc.).
Cada dado é associado a um rótulo (nome simbólico) para facilitar o acesso.
Utiliza diretivas como `.word` e `.asciiz`.

SEÇÃO DE CÓDIGO

Inicia com a diretiva `.text`.
Contém as instruções que serão executadas pelo processador.
O rótulo `main`: identifica o ponto de entrada do programa.
Aqui ficam as instruções, pseudoinstruções, chamadas de sistema e demais rótulos utilizados nos desvios e saltos.

RÓTULOS IMPORTANTES

- `main`: início da execução do programa
- Outros rótulos: usados para desvios (`beq`, `bne`) e saltos (`j`).

VISÃO GERAL DA ESTRUTURA



Observação: Comentários iniciam com `#` e vão até o final da linha.

2. Comentários

Comentários são utilizados para documentar o código-fonte e não fazem parte das instruções executadas pelo processador. Na μ -Assembly, um comentário é iniciado pelo caractere `#`. Todo o conteúdo localizado entre `#` e o final da linha é considerado comentário.

Exemplo:

```
li $t0, 10      # carrega o valor 10
```

Também é permitido utilizar uma linha exclusivamente para comentários:

```
# Inicialização dos registradores  
li $t0, 10
```

No entanto, o caractere `#` localizado no interior de uma string não representa o início de um comentário, como pode ser observado em msg: `.asciiz "Resultado #1"`. Nesse caso, `#1` pertence à string.

3. Identificadores

Um **identificador** é um nome definido no código-fonte para *identificar simbolicamente algum elemento do programa*. Em vez de o programador trabalhar diretamente com endereços numéricos ou outras representações internas, ele utiliza nomes que tornam o código mais compreensível e fácil de manter. No contexto de **Assembly MIPS**, os identificadores aparecem principalmente na definição de **rótulos**. Considere:

```
.data
mensagem: .asciiz "Resultado: "
numero: .word 10

.text
main:
    lw $t0, numero
```

Nesse exemplo, mensagem, numero e main são identificadores. O caractere : não faz parte do identificador e indica que aquele identificador está sendo *declarado como um rótulo*.

Identificador e rótulo não são exatamente a mesma coisa

É útil fazer essa distinção. Identificador é o nome propriamente dito; rótulo é o símbolo definido por esse nome e associado a uma determinada posição.

Por exemplo, em *inicio:*, inicio é o identificador, enquanto inicio: constitui a declaração de um rótulo. Posteriormente, podemos utilizar esse identificador em instruções como *j inicio*. Nesse caso, inicio não está sendo declarado novamente. Ele está sendo referenciado.

Tem-se, portanto:

- inicio: é a declaração do rótulo
- j inicio é a referência ao rótulo

Regras para formação dos identificadores

Uma linguagem precisa estabelecer quais sequências de caracteres podem formar identificadores. No μ -Assembly, um identificador deve começar por uma letra ou pelo caractere _ e pode ser seguido por letras, algarismos ou _.

Com essa regra, inicio, loop1, soma_total e _inicio são exemplos válidos, diferentemente de 1inicio, soma-total e minha soma, que violam alguns desses preceitos. Formalmente, podemos descrever identificadores como

$$\text{IDENTIFICADOR} \rightarrow (\text{LETRA} \mid \text{"_"})(\text{LETRA} \mid \text{DÍGITO} \mid \text{"_"})^*$$

ou, usando uma expressão regular, `[A-Za-z_][A-Za-z0-9_]*`

O papel do analisador léxico

Quando o analisador léxico encontra:

loop:

ele pode separar essa entrada em tokens como

IDENTIFICADOR("loop")

DOIS_PONTOS(":")

Já em:

j loop

poderíamos ter:

INSTRUCAO("j")

IDENTIFICADOR("loop")

Nesse momento, o analisador léxico apenas reconhece que loop possui a forma válida de um identificador. Ele não precisa determinar ainda se existe realmente um rótulo chamado loop. Essa distinção é importante no projeto do compilador.

Por exemplo:

j destino

destino pode ser um **identificador lexicalmente válido**, mesmo que não exista:

destino:

em nenhum lugar do programa. A forma do identificador está correta; o problema é que ele referencia um símbolo inexistente. Essa verificação será feita posteriormente, durante a **análise semântica**, com auxílio da **tabela de símbolos**.

Em resumo:

- **identificador** é o nome utilizado para representar simbolicamente um elemento;
- **rótulo** é a associação desse nome a uma posição do programa ou dos dados;
- **tabela de símbolos** é a estrutura utilizada pelo assembler para registrar essas associações.

4. Rótulos

No Assembly MIPS, um rótulo (*label*) é um nome simbólico associado a uma determinada posição do programa, seja na memória de dados, seja na memória de instruções. Seu principal objetivo é permitir que o programador faça referência a endereços utilizando nomes significativos, sem precisar conhecer ou escrever diretamente os respectivos endereços numéricos.

Por exemplo:

```
.data
mensagem: .asciiz "Olá: "
numero: .word 10

.text
main:
    lw $t0, numero
```

Nesse programa, mensagem, numero e main são identificadores utilizados como rótulos. O assembler associa cada um deles a um determinado endereço. Conceitualmente, mensagem é o endereço onde começa "Olá!", numero é o endereço onde está armazenado 10 e main é o endereço da primeira instrução do programa

Declaração de um rótulo

Normalmente, um rótulo é declarado colocando-se um identificador seguido de dois-pontos:

início:

Nesse caso:

- início é o identificador
- : indica a definição do rótulo
- início: corresponde à declaração do rótulo

O rótulo não representa uma instrução e, em geral, não ocupa espaço próprio na memória. Ele simplesmente atribui um nome ao endereço correspondente à posição em que foi declarado.

Considere:

```
.text
inicio:
    li $t0, 0
    addi $t0, $t0, 1
```

Se, hipoteticamente, a instrução `li` estivesse localizada no endereço `0x00400000`, o assembler poderia registrar início como `0x00400000`. Ou seja, início: não gera necessariamente uma instrução de máquina; ele identifica a posição da instrução que o sucede.

Rótulos na seção `.text`

Na seção de código, os rótulos são muito utilizados para identificar **pontos para os quais a execução poderá ser desviada**.

Por exemplo:

```
.text
main:
    li $t0, 0
    li $t1, 10
loop:
    addi $t0, $t0, 1
    bne $t0, $t1, loop
fim:
```

Nesse exemplo, temos três rótulos: `main`, `loop` e `fim`. A instrução `bne $t0, $t1, loop` utiliza o rótulo `loop` como destino do desvio. Enquanto `$t0` for diferente de `$t1`, a execução retorna à posição associada a `loop`. Sem rótulos, seria necessário trabalhar diretamente com endereços ou deslocamentos, tornando o código muito menos legível.

Rótulos na seção `.data`

Os rótulos também podem identificar posições destinadas ao armazenamento de dados:

```
.data
mensagem: .asciiz "Olá: "
idade: .word 20
```

Nesse caso, `idade` identifica o endereço onde o valor `20` está armazenado, enquanto `mensagem` identifica o endereço inicial da sequência de caracteres `"Olá!"`. Isso permite escrever `lw $t0, idade` em vez de informar diretamente um endereço de memória.

Portanto, podemos distinguir dois usos bastante comuns:

Local	Exemplo	O que o rótulo identifica
<code>.text</code>	<code>loop:</code>	posição de uma instrução
<code>.data</code>	<code>numero:</code>	posição de um dado

Declaração e referência

É importante diferenciar a declaração de um rótulo de sua referência.

Em:

```
loop:
    addi $t0, $t0, 1
    j loop
```

temos em **loop:** uma declaração e em **j loop** uma referência. Na declaração, o assembler precisa descobrir qual endereço deverá ser associado ao símbolo **loop**. Na referência, precisa descobrir qual endereço está associado ao nome utilizado.

Essa associação é registrada na tabela de símbolos:

Símbolo	Endereço
main	0x00400000
loop	0x00400008
fim	0x00400010
numero	0x10010000

Por que a tabela de símbolos é necessária?

Para ilustrar a importância da tabela de símbolos, considere:

```
.text
main:
    j fim

    add $t0, $t1, $t2

fim:
    li $v0, 10
```

Quando o assembler encontra **j fim**, o rótulo **fim** ainda aparece mais adiante no arquivo. Essa situação é chamada de referência futura (*forward reference*). O assembler precisa ser capaz de descobrir posteriormente o endereço correspondente a **fim** e utilizar essa informação para gerar corretamente a instrução de salto.

Esse problema ajuda a explicar por que assemblers são frequentemente apresentados didaticamente como tradutores que podem trabalhar em duas passagens pelo código: uma passagem determina os endereços e constrói a tabela de símbolos; outra utiliza essas informações para resolver as referências e gerar o código.

Erros relacionados aos rótulos

Na implementação de um assembler, vários erros semânticos podem estar associados aos rótulos. Por exemplo, uma referência a um rótulo inexistente:

- **j destino** sem que exista **destino:** deverá ser detectada.
- Também podemos ter uma declaração duplicada.

Como o mesmo símbolo não deveria representar simultaneamente duas posições diferentes nesse escopo, o assembler deverá sinalizar o problema.

Uma definição mais completa

No contexto do projeto, considere a seguinte definição:

Rótulo é um símbolo definido por meio de um identificador e associado a uma posição do código ou dos dados, permitindo que essa posição seja referenciada simbolicamente em outras partes do programa. Durante a montagem, o assembler determina o endereço correspondente ao rótulo, registra essa associação na tabela de símbolos e utiliza-a para resolver as referências existentes no código.

5. Diretivas

As diretivas são comandos destinados ao assembler, usados para orientar a organização e a montagem do programa. Diferentemente das instruções MIPS, elas não representam operações que a CPU executa diretamente. Em geral, servem para definir seções, reservar ou inicializar dados, estabelecer alinhamento, declarar símbolos e fornecer outras informações necessárias ao processo de montagem.

Por convenção, muitas diretivas utilizadas no MARS começam com um ponto (.):

`.data`

`.text`

`.word`

`.ascii`

`.globl`

É importante, portanto, distinguir a instrução **add \$t0, \$t1, \$t2** da diretiva **.word 10**. Na primeira linha, **add** representa uma operação que será traduzida para uma instrução de máquina MIPS. Na segunda, **.word** fornece ao assembler uma orientação sobre como um dado deverá ser armazenado na memória.

Diretivas `.data` e `.text`

As duas diretivas mais importantes para compreender a estrutura de um programa são **.data** e **.text**. A diretiva **.data** indica o início da seção de dados. Nessa região normalmente são declaradas variáveis, constantes, vetores e strings.

Por exemplo:

```
.data
mensagem: .ascii "Olá, mundo!"
idade: .word 25
```

Já **.text** indica a seção destinada ao código, isto é, às instruções que serão executadas:

```
.text
main:
    li $t0, 10
    li $t1, 20
    add $t2, $t0, $t1
```

Assim, podemos visualizar em **.data** dados, variáveis, strings e vetores; e em **.text** instruções, desvios, saltos e chamadas.

Diretiva **.word**

A diretiva **.word** permite armazenar valores como palavras na memória. Assim, em **numero: .word 10**, o assembler reserva espaço para uma palavra e a inicializa com o valor 10. O rótulo **numero** fica associado ao endereço correspondente. Também é possível declarar uma sequência de valores como em vetor: **.word 10, 20, 30, 40**

Diretivas **.byte** e **.half**

Existem diretivas para armazenar dados de outros tamanhos. Por exemplo:

valor: **.byte 65**

reserva um **byte**.

Já:

valor: **.half 1000**

armazena um valor de **meia palavra** (*halfword*).

Em uma visão simplificada do MIPS:

Diretiva	Tamanho típico
.byte	1 byte
.half	2 bytes
.word	4 bytes

Essas diretivas são interessantes quando o assembler precisa calcular os endereços dos rótulos: o tamanho do dado declarado influencia a posição do próximo elemento.

Diretivas **.ascii** e **.asciiz**

São utilizadas para armazenar sequências de caracteres.

Por exemplo:

- **mensagem: .ascii "Olá"** armazena os caracteres da string.
- **mensagem: .asciiz "Olá"** armazena a sequência e acrescenta um **caractere nulo (\0)** ao final.

Conceitualmente, **.ascii "ABC"** lida com os valores 'A' 'B' 'C' (65 66 67) enquanto **.asciiz "ABC"** com os valores 'A' 'B' 'C' '\0' (65 66 67 0). O terminador nulo é importante porque permite identificar onde a string termina.

Diretiva **.space**

A diretiva **.space** é usada para reservar determinada quantidade de bytes na memória. Por exemplo, **buffer: .space 100** reserva 100 bytes, que poderão ser utilizados posteriormente pelo programa. Ela é útil para vetores, buffers e áreas destinadas à entrada de dados.

Diretiva **.align**

A diretiva **.align** permite controlar o alinhamento dos dados na memória. O alinhamento é importante porque determinados tipos de dados devem, ou é conveniente que, comecem em endereços compatíveis com seu tamanho. Em MIPS/MARS, o argumento de **.align** deve ser interpretado conforme a regra específica adotada pelo assembler; por exemplo, **.align 2** normalmente conduz ao alinhamento em uma fronteira de bytes.

Para um assembler didático, essa diretiva pode ser deixada para uma etapa mais avançada, porque exige que os alunos tratem também o cálculo e ajuste dos endereços.

Diretiva **.globl**

A diretiva **.globl main** declara **main** como um símbolo global. Um programa pode apresentar, por exemplo:

```
.text
.globl main

main:
    li $v0, 10
    syscall
```

A diretiva **.globl** fornece informação sobre a visibilidade do símbolo; ela não corresponde a uma instrução executada pela CPU.

Diretivas podem afetar a memória sem gerar instruções

Essa característica é importante para o projeto do assembler.

Considere:

```
.data

x: .word 10
y: .word 20

.text

main:
    add $t0, $t1, $t2
```

Nesse Código, **.data**, **.word** e **.text** não precisam produzir instruções MIPS. Entretanto, elas têm efeitos importantes sobre a montagem. Por exemplo, **x: .word 10** determina que o assembler deverá colocar o valor 10 na região de dados e associar ao rótulo x o endereço correspondente.

Atenção:

As diretivas não representam instruções executadas diretamente pela CPU, embora possam fazer com que o assembler reserve ou inicialize posições de memória e produza informações no arquivo resultante.

Diretiva × instrução × pseudoinstrução

É importante destacar a diferença entre diretiva, instrução e pseudoinstrução.

Elemento	Exemplo	Função
Diretiva	.word 10	orienta o assembler
Instrução	add \$t0,\$t1,\$t2	representa operação da CPU
Pseudoinstrução	li \$t0,10	forma conveniente traduzida pelo assembler

Veja:

```
.data                                # DIRETIVA
numero: .word 10                    # RÓTULO + DIRETIVA

.text                                # DIRETIVA
main:                                # RÓTULO
    li $t0, 10                      # PSEUDOINSTRUÇÃO
    add $t1, $t0, $t0               # INSTRUÇÃO
```

Essa distinção será muito importante na **análise léxica** do projeto. O lexer poderá, por exemplo, produzir categorias diferentes:

- DIRETIVA(".data")
- IDENTIFICADOR("numero")
- DOIS_PONTOS
- DIRETIVA(".word")
- INTEIRO("10")
- DIRETIVA(".text")
- IDENTIFICADOR("main")
- DOIS_PONTOS
- PSEUDOINSTRUCAO("li")
- REGISTRADOR("\$t0")
- VIRGULA
- INTEIRO("10")

Depois, a análise sintática poderá verificar se cada diretiva está sendo utilizada de acordo com a estrutura definida para a linguagem.

6. Strings

Uma **string** corresponde a uma sequência de caracteres delimitada por aspas duplas.

Exemplo:

"Resultado:"

Uma declaração completa pode ser **msg: .asciiz "Resultado:"**. Os espaços existentes dentro da string fazem parte de seu conteúdo. Assim, **msg: .asciiz "A B"** possui significado diferente de **msg: .asciiz "A B"**. O caractere # também poderá aparecer dentro de uma string **msg: .asciiz "Resultado #1"**. Nesse contexto, ele não será interpretado como comentário.

7. Valores numéricos e registradores

Valores numéricos

A μ -Assembly deverá reconhecer valores inteiros como 0, 1, 10, 25 e 100. Também poderão ser utilizados valores negativos como -1, -10 e -100. São exemplos de utilização `li $t0, 10`; `li $t1, -5` e `valor: .word 100`

Registradores

Os registradores são pequenas unidades de armazenamento localizadas no processador e utilizadas diretamente pelas instruções. Neste projeto, serão considerados os seguintes registradores MIPS.

1	Registradores temporários	\$t0, \$t1, \$t2, \$t3, \$t4, \$t5, \$t6, \$t7, \$t8 e \$t9 são normalmente utilizados para armazenar valores temporários durante os cálculos.
2	Registradores de valores	\$v0 e \$v1 são utilizados, entre outras finalidades, para valores de retorno e chamadas de sistema.
3	Registradores de argumentos	\$a0, \$a1, \$a2 e \$a3 são normalmente empregados na passagem de argumentos.
4	Registrador zero	\$zero é o registrador que representa o valor constante zero (0). Para utilizá-lo, basta usar <code>add \$t0, \$t1, \$zero</code>

Separadores

Os elementos de uma instrução podem ser separados por espaços, enquanto os operandos são separados por vírgulas.

Exemplo:

`add $t0, $t1, $t2`

Nesse caso:

- `add` é a operação;
- `$t0` é o primeiro operando;
- `,` é um separador;
- `$t1` é o segundo operando;
- `$t2` é o terceiro operando.

Os dois-pontos (`:`) são utilizados para identificar o término de um rótulo:

`main:`

As aspas (`"`) delimitam strings:

`"Olá!"`

Instruções aritméticas

Nº	Instrução	Descrição	Sintaxe
1	add	Realiza a soma dos valores armazenados em dois registradores.	add <destino>, <origem1>, <origem2>
2	sub	Realiza a subtração dos valores armazenados em dois registradores.	sub <destino>, <origem1>, <origem2>
3	and	Realiza a operação lógica AND bit a bit entre dois registradores.	and <destino>, <origem1>, <origem2>
4	or	Realiza a operação lógica OR bit a bit entre dois registradores.	or <destino>, <origem1>, <origem2>
5	xor	Realiza a operação lógica XOR bit a bit entre dois registradores.	xor <destino>, <origem1>, <origem2>
6	mul	Realiza a multiplicação dos valores armazenados em dois registradores e armazena o resultado no registrador de destino.	mul <destino>, <origem1>, <origem2>
7	div	Realiza a divisão inteira entre dois registradores. O quociente e o resto são armazenados em registradores especiais.	div <origem1>, <origem2>

No caso da divisão tradicional do MIPS, a sintaxe é diferente das demais. Em **div \$t1, \$t2**, o resultado é armazenado em dois registradores especiais: **LO**, que armazena o quociente e **HI** que armazena o resto.

Para recuperar os valores:

```
div $t1, $t2
mflo $t0      # $t0 recebe o quociente
mfhi $t3      # $t3 recebe o resto
```

Por exemplo, se \$t1 = 17 e \$t2 = 5, após a divisão teremos LO = 3 e HI = 2.

8. Pseudoinstrução li

A pseudoinstrução li (**load immediate**) permite carregar um valor inteiro em um registrador.

Formato:

li <registrador>, <valor>

Exemplo:

li \$t0, 10

Após essa operação, \$t0 conterá o valor 10.

Outro exemplo:

li \$t1, -5

9. Acesso à memória

9.1. lw

A instrução lw (**load word**) permite carregar uma palavra da memória para um registrador.

Exemplo:

```
lw $t0, 0($t1)
```

9.2. sw

A instrução sw (**store word**) permite armazenar na memória o conteúdo de um registrador.

Exemplo:

```
sw $t0, 0($t1)
```

Essas instruções introduzem a seguinte forma de endereçamento, ou seja, **deslocamento(registrador)**. Por exemplo, 4(\$t0).

10. Desvios condicionais

10.1. beq

A instrução beq (**branch if equal**) realiza um desvio quando dois registradores possuem valores iguais.

Formato:

beq <reg1>, <reg2>, <rotulo>

Exemplo:

beq \$t0, \$t1, fim

10.2. bne

A instrução bne (**branch if not equal**) realiza um desvio quando os registradores possuem valores diferentes.

bne <reg1>, <reg2>, <rotulo>

Exemplo:

inicio:

add \$t0, \$t0, \$t1

bne \$t0, \$t2, inicio

11. Saltos

11.1. j

A instrução j realiza um salto incondicional para um rótulo.

Formato:

j <rotulo>

Exemplo: j fim

Programa:

```
.text
main:
    li $t0, 10
    j fim
inicio:
    li $t0, 20
fim:
```

12. Chamadas de sistema

No MARS, determinadas operações de entrada e saída podem ser realizadas por meio de chamadas de sistema utilizando:

syscall

O código da operação é normalmente informado no registrador \$v0.

Por exemplo, uma estrutura típica é:

li \$v0, <codigo>

syscall

Os códigos de serviços efetivamente aceitos pela **μ-Assembly deverão ser definidos como parte da especificação do subconjunto da linguagem**, caso chamadas de sistema sejam incluídas no projeto.

13. Elementos léxicos da μ -Assembly

Considerando as construções apresentadas neste manual, o código-fonte poderá conter, entre outros, os seguintes elementos:

Categoria	Exemplos
Diretivas	.data, .text, .word, .asciiiz
Instruções	add, sub, and, or, xor, lw, sw, beq, bne, j
Pseudoinstruções	li
Registradores	\$t0, \$t1, \$v0, \$a0, \$zero
Identificadores	main, inicio, mensagem
Rótulos	main:, inicio:, fim:
Inteiros	10, 0, -25
Strings	"Resultado: "
Separadores	,, :, (,)
Comentários	# comentário

Essa classificação será especialmente importante na etapa de **análise léxica**, na qual o fluxo de caracteres produzido pelo pré-processador será transformado em uma sequência de tokens.